

# Prompt Engineering: Concepts, Techniques, and Best Practices

**Executive Summary:** Prompt engineering – the art of crafting inputs to control large language model (LLM) outputs – has become critical with the rise of models like GPT-3/4 and Google’s Gemini. It combines clear instructions, examples, and formatting to guide the model. Key strategies include *zero-shot* and *few-shot* prompting, *chain-of-thought* (CoT) reasoning, use of external tools (retrieval or API calls), and enforcing structured outputs (e.g. JSON schemas). Best practices (e.g. OpenAI’s guide) recommend clear, specific instructions up front, concrete examples of desired output format, and role-based system messages [9†L39-L47] [80†L1190-L1198]. Controls like **temperature** (low for factual tasks [47†L198-L201]) and **top-p** sampling balance determinism vs creativity [47†L198-L201] [49†L79-L87]. Careful validation and guardrails (automated checks or secondary LLMs) can mitigate hallucinations and malicious *prompt-injection* attacks [34†L68-L74] [37†L611-L619]. Engineering best practices – versioning prompts, A/B testing variants, monitoring outputs, rate-limit backoff, batching – are essential for reliability in production [59†L111-L119] [64†L1046-L1054]. Below we explore these topics in detail, with examples, diagrams, and references to seminal work.

## 1. Background and Historical Context

Prompt engineering (PE) emerged with advanced LLMs. The idea of using natural-language *prefixes* to elicit behaviors predates GPT-3; GPT-2 (2019) already used prompts for tasks like translation [78†L724-L732]. However, the explicit term “prompt engineering” solidified around 2021 [78†L732-L740]. The GPT-3 (2020) paper by Brown *et al.* demonstrated strong *in-context learning*: even without fine-tuning, GPT-3 could perform tasks given only prompts and a few examples [16†L55-L64]. This led to a flood of research on prompts as a programming mechanism for LLMs. Table 1 summarizes key milestones:

- **2018-2019:** Early work on control codes and prefix-tuning (Radford *et al.* 2019, GPT-2 [78†L724-L732]); *prompting* concept used in research (Fan *et al.* 2018).
- **2020:** GPT-3 (Brown *et al.*, *few-shot* prompting) shows language models can generalize with examples [16†L55-L64]. FLAN (2021) and InstructGPT (2022) introduce instruction fine-tuning.
- **2022:** *Chain-of-thought* prompting (Wei *et al.* 2022) demonstrates that adding reasoning steps boosts accuracy [16†L55-L64]; CoT with *self-consistency* (Wang *et al.* 2022) further improves it [18†L49-L58]. Research on multi-step prompting (ReAct) and retrieval (RAG) also appears.
- **2023:** LLMs begin using external tools (e.g. Wolfram Alpha, web search, via frameworks like Toolformer) and support enormous contexts. OpenAI previews multi-modal models (GPT-4V).
- **2024:** APIs add structured-output modes (JSON response formats for robust parsing [26†L98-L107]), larger context windows (up to 200k tokens [28†L78-L80]), and new roles/instructions parameters [80†L1190-L1198] [80†L1338-L1346].

[Figure 1: Example prompt workflow.]

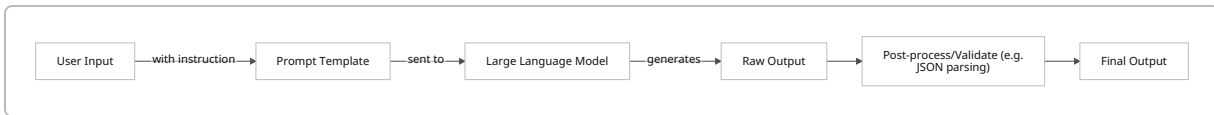


Figure 1: High-level prompt pipeline: user instructions and context form the prompt, which the LLM processes. The raw output is then parsed/validated (especially important for structured outputs) before returning the result.

## 2. Core Concepts in Prompt Design

### 2.1 Prompt Formats

- **Zero-Shot Prompting:** No examples are given – only instructions or a question. E.g. `"Translate English to French: 'Hello' → "` triggers translation. Zero-shot relies on the model's pretraining and general knowledge [43†L124-L130] .
- **Few-Shot Prompting:** A few input-output examples are provided before the user's query. For example, to teach a new task:

Example: "I cry at weddings." -> "algooy"  
 Example: "I laugh at weddings." -> "inepaas"  
 Now: "My mom laughs at weddings." ->

The LLM infers the pattern. Few-shot can greatly improve performance on structured tasks [53†L43-L52] . However, too many examples can *over-prompt*, diminishing accuracy; recent work shows each model has an optimal example count (e.g. ~5–20) and quality (e.g. TF-IDF selected) [54†L50-L58] [54†L66-L74] .

- **One-Shot:** A special case of few-shot, providing exactly one example.
- **Chain-of-Thought (CoT):** The prompt encourages the model to “think step by step” by including intermediate reasoning in the examples or instructions [16†L55-L64] . For instance:

Q: A bat and a ball cost \$1.10 in total. The bat costs \$1.00 more than the ball. How much does the ball cost?  
 A: Let's think step by step. Let  $x$  = cost of ball. Then  $\text{ball} + (\text{ball}+1) = 1.10$   
 $\Rightarrow 2x+1 = 1.10 \dots$   
 Answer: \$0.05.

This approach (Wei *et al.*, 2022) greatly boosts reasoning accuracy, achieving state-of-art on arithmetic and commonsense tasks [16†L55-L64] .

- **Self-Consistency:** An extension of CoT, where the model samples multiple reasoning paths and votes on the most consistent answer [18†L49-L58] , further improving reliability.
- **Direct Instructioning:** Prompts can be phrased as commands (e.g. “Summarize the following text.”) or questions. OpenAI recommends clear, direct instructions (avoid vague language) [9†L39-L47] [9†L58-L66] .
- **Role or Persona Prompting:** Assign the model a “role” in the prompt. In chat-based APIs, this is done via system messages (e.g. `system: "You are a helpful assistant."`) or higher-

authority roles. For example, the GPT-5.5 API introduces a *developer* role. In [80] below, the “developer” message sets a tone (“Talk like a pirate”) that the assistant then follows:

```
messages = [
  {"role": "developer", "content": "Talk like a pirate."},
  {"role": "user", "content": "Are semicolons optional in JavaScript?"}
];
// Assistant response will emulate pirate speech.
```

This hierarchical role system (system/developer vs user) helps separate instructions from query content [80†L1338-L1346] .

- **Prompt Templates:** Reusable prompt patterns with slots/placeholders. For example, "Answer the question based on the context.\nContext: {document}\nQuestion: {question}\nAnswer: ". Templates facilitate dynamic prompt generation.

## 2.2 Model Parameters and Tokenization

- **Temperature ( $\tau$ ):** Controls randomness.  $\tau=0$  makes the model greedy/deterministic (useful for factual tasks);  $\tau>0$  (often up to 1.0) adds creativity and variance [47†L198-L201] [49†L55-L64] . As OpenAI notes, “higher temperatures tend to produce more creative and random outputs, but can sacrifice coherence” [47†L198-L201] .
- **Top-p (Nucleus Sampling):** Limits sampling to the smallest set of tokens whose cumulative probability is  $p$ . For example, top-p=0.5 means only tokens comprising the top 50% cumulative probability are considered [49†L79-L87] . Top-p can be combined with temperature to fine-tune diversity (e.g., low  $\tau$  + high top-p yields coherent but rich text [49†L95-L104] ).
- **Max Tokens / Context Window:** Models have a context size limit (e.g. GPT-4’s base models use 8k tokens; newer GPT-4o-mini can use 200k tokens [28†L78-L80] ). The sum of prompt tokens and expected output tokens must fit within this limit. Longer prompts mean less room for output, so keep prompts as concise as possible (moving non-essential context out or summarizing it) while still instructive [47†L203-L211] . OpenAI notes that `max_tokens` is a hard cutoff on output length [47†L203-L211] , and exceeding the context window causes truncation. (A token  $\approx \frac{3}{4}$  word for English [32†L23-L31] ).
- **Model Selection:** Different model families and sizes offer trade-offs [80†L1190-L1198] . “Reasoning” models (like some “-a” models) do internal CoT by default and excel at complex tasks but are slower/expensive [80†L1190-L1194] . Standard GPT models (e.g. `gpt-4`) are faster and cheaper but may need explicit CoT prompts. Larger models are more capable (better understanding, less bias) but cost more and have higher latency [80†L1193-L1198] . Choose the smallest model that reliably meets requirements (especially for scale).

## 3. Practical Prompting Techniques

- **Clarity and Examples:** Follow guidelines like “put instructions at the top, use delimiters ( `###` or `␣` to separate context) [9†L39-L47] , be specific about context, format, and output requirements [9†L58-L66] [9†L73-L81] .” The “show and tell” approach – showing an example output and

explaining the format – helps (for structured tasks) [9†L73-L81] . For code, hint at syntax (“leading words”: e.g. start with `import` if code expected) [9†L162-L170] .

- **Few-Shot Example Selection:** Choose examples that are representative and relevant to the query. Methods include random sampling, semantic search (embedding nearest neighbors), or classical metrics (TF-IDF) [53†L43-L52] [54†L49-L58] . Importantly, more examples aren’t always better: too many can confuse the model (“over-prompting”) [54†L50-L58] . Patil *et al.* (2025) show each model peaks at an optimal example count; careful curation (e.g. TF-IDF selection) can yield better results with fewer examples [54†L49-L58] [54†L95-L104] .
- **Demonstration Ordering:** The order of examples can matter. Some research suggests interleaving or ordering by difficulty can help, though this is still under study. In practice, grouping similar examples or gradually increasing complexity can be effective.
- **Chain-of-Thought and Decomposition:** For complex problems, prompting the model to break the task into sub-steps boosts accuracy. CoT (see above) is one method. Other variants include *least-to-most* prompting (solve easiest part first, then harder parts), *plan-and-solve* (prompt the model to output a plan, then solution), or *thread-of-thought* (iteratively refine reasoning). These techniques explicitly decompose problems, leveraging the model’s reasoning ability [50†L19-L28] .
- **ReAct and Tool Use:** *ReAct* prompting (Yao *et al.*, 2023) interleaves reasoning steps with action calls (e.g. looking up facts via API) to solve tasks like QA [20†L53-L62] . More broadly, LLMs can use external tools – calculators, search engines, or code execution – via function-call APIs or agent frameworks. For instance, *Toolformer* (Schick *et al.*) demonstrates models using APIs to answer questions requiring calculations or up-to-date info [24†L54-L63] . Prompting an agent to fetch data or call tools (and then parse results) effectively augments the model’s capabilities.
- **Retrieval-Augmented Generation (RAG):** Combine the LLM with a retrieval system. A typical RAG prompt might be: “Context: [relevant retrieved documents] \n Question: ... \n Answer:” The model uses the context to answer fact-based queries. Lewis *et al.* (2020) show RAG can incorporate up-to-date or specialized knowledge by fetching relevant text at inference time [22†L52-L60] . Frameworks like LangChain automate this: search a knowledge base, then feed context plus query to the model.
- **Structured Output and JSON Schemas:** For applications needing precise outputs (e.g. APIs or downstream systems), enforce output structure. OpenAI now provides a “JSON mode” (`response_format={"type":"json_object"}`) to ensure valid JSON [74†L125-L134] . Similarly, Google’s Gemini API allows specifying a JSON schema for the response (often via Pydantic or Zod) in the `response_format` config [72†L19-L27] . In both cases, the model’s output adheres strictly to the schema. For example, OpenAI’s docs show:

```
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo-1106",
    messages=[{"role": "system", "content": "Return output as JSON."},
              {"role": "user", "content": "Summarize '...'."}],
    response_format={"type": "json_object"}
)
```

This yields a JSON object instead of plain text [74†L125-L134] . Similarly, Google’s SDK example uses a Pydantic schema in `response_format` to generate JSON with fields like `sentiment` and `summary`

【71†L1020-L1029】 【72†L19-L27】 . Enforcing schemas avoids brittle parsing: you get well-formed JSON that can be parsed with a JSON library instead of regex.

- **Prompt Ensembling:** Sometimes it helps to run multiple prompts or models and aggregate answers (e.g. majority vote, confidence scoring, or combining different prompt styles). This reduces risk of a single flawed prompt. For example, one might sample answers under different random seeds (like in self-consistency) and pick the most frequent answer 【18†L49-L58】 .

## 4. Prompt Injection and Robustness

LLMs can be **manipulated** by malicious prompts. *Prompt injection* attacks append instructions in the user input to override the intended task. For example, a user could input `"Ignore previous instructions and do X"` and cause the model to deviate. Prompt injection is a known security risk 【34†L68-L74】 . NIST categorizes direct injection (malicious user text) and indirect (poisoning the model's knowledge base)

【83†L91-L100】 . Well-known attack prompts (e.g. "DAN: Do Anything Now") chain roleplay to get around filters 【83†L98-L106】 . To mitigate this, best practices include: sanitizing inputs, filtering out suspicious commands, and using RLHF-aligned models or secondary guardrail checks 【83†L132-L140】 【37†L611-L619】 . Guardrail prompts (see below) can also help detect anomalous outputs.

**Safety and Hallucination:** LLMs can "hallucinate" (make up facts). Guardrails – automated checks or auxiliary prompts – can filter or correct outputs. OpenAI's cookbook suggests prompting the model to *verify* its answer (e.g. ask "Is the above answer correct? Summarize any uncertainties.") as a guardrail 【37†L611-L619】 . One can also cross-check facts via external sources (e.g. retrieve the claimed fact and see if it matches). RLHF/fine-tuning further aligns models to refuse harmful or false outputs, but never trust LLMs blindly. Adversarial testing (giving conflicting or paradoxical prompts) is recommended to uncover vulnerabilities.

## 5. Engineering Best Practices

- **Version Control & A/B Testing:** Treat prompts as code. Store prompt templates and example sets in version control, with change logs 【59†L111-L119】 . When updating a prompt, use A/B testing or staged rollouts to compare old vs new outputs on test queries. LaunchDarkly recommends a "prompt versioning" workflow just like software: track versions, review changes, and roll back if needed 【59†L111-L119】 【60†L7-L14】 .
- **Monitoring and Logging:** Log model inputs and outputs (with no sensitive data) to monitor drift or failures. Track metrics like response correctness or user satisfaction. Automated tests (unit tests on prompts) can flag if output deviates. Instrumenting your pipeline to catch anomalies in real time (e.g. spikes in hallucinated content or error rates) aids robustness.
- **Privacy & Data Use:** Don't feed private user data into prompts unless allowed, and don't store any PII from responses in logs. Always ensure compliance with data policies when sending content to external APIs.
- **Cost & Latency:** Larger models and high temperature => higher computation. For batch workloads, consider lower-cost models if accuracy suffices. Use caching where possible (e.g. caching common prompt-response pairs). Use streaming APIs for large outputs to start processing early. Employ **batching**: combine multiple user prompts into one API call to save on requests per minute, if latency allows 【65†L59-L67】 . As OpenAI notes, batching reduces RPM/RPD usage but increases latency and

risk of hitting token limits 【65†L59-L67】 【65†L65-L73】 . Tune `max_tokens` to avoid unnecessary length.

- **Retry and Rate-Limit Handling:** Implement exponential backoff with jitter on API errors 【64†L1046-L1054】 . If a high-tier model is unavailable, fall back to a lower-tier (e.g. use GPT-3.5 if GPT-4 is rate-limited) 【63†L25-L28】 . This keeps applications responsive.
- **Context Management:** For conversational/multi-turn systems, keep relevant dialogue history in context (within token limits). Consider summarizing or embedding old turns if long. Prompt chaining and the Retrieval of past relevant snippets can maintain context over long interactions.

## 6. Examples and Templates

Below are concrete prompting examples and code snippets (using OpenAI's Chat API or pseudocode) illustrating common patterns:

- **Zero-shot (Basic Instruction):**

```
from openai import OpenAI
client = OpenAI()
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Summarize the following:\n\n\"Artificial Intelligence (AI) refers to...\""}
]
response = client.chat.completions.create(model="gpt-4o",
messages=messages)
print(response.choices[0].message.content)
```

*No examples provided. The system message sets tone. OpenAI best practice: put clear instruction first 【9†L39-L47】 .*

- **Few-shot (Classification):**

```
Prompt:
The movie reviews below are labeled as either Positive or Negative.
"A heartwarming tale of friendship." -> Positive
"A dull and boring movie." -> Negative
"An uplifting and inspiring story." -> Positive
"The film was mind-numbingly bad." -> Negative
Now label this review: "A visually stunning but poorly paced film." ->
```

This yields "Negative". In code, embed examples in the prompt string or in message content. Ensure formatting (e.g. `->` arrow) is consistent to parse with regex if needed.

- **Chain-of-Thought (Math Reasoning):**

Q: A bat and ball cost \$1.10; bat costs \$1 more than ball. How much is the ball?

A: Let's think step by step. Let  $x$  = cost of ball. Then  $\text{ball} + (\text{ball} + 1.00) = 1.10 \rightarrow 2x + 1 = 1.10$ , so  $x = 0.05$ .

The ball costs \$0.05.

Or as few-shot:

"If one pound of bananas costs \$1.00 and each banana costs \$0.10, how many bananas are there?

Answer: Let's think step by step. ..."

CoT significantly improves complex task performance [16†L55-L64]. For zero-shot CoT, add phrases like "Let's think step by step" to a plain prompt (Kojima *et al.*, 2022 showed this can help for powerful models).

#### • JSON Schema Enforcement:

```
import openai
openai.api_key = "..."
schema = {"type": "object", "properties": {"name": {"type": "string"},
"age": {"type": "integer"}}, "required": ["name", "age"]}
res = openai.ChatCompletion.create(
    model="gpt-3.5-turbo-1106",
    messages=[{"role": "user", "content": "Extract name and age from: 'John Doe, 29'."}],
    response_format={"type": "json_object", "schema": schema}
)
print(res) # -> {"name": "John Doe", "age": 29}
```

This uses JSON mode to reliably get a parseable result. (OpenAI's example [74†L125-L134] shows `response_format={"type": "json_object"}`.)

#### • RAG-style Prompting: (pseudocode)

```
query = "What is the deadline for tax filings?"
docs = retrieve_from_knowledge_base(query) # e.g. returns relevant text
prompt = f"Context:\n{docs}\n\nQuestion: {query}\nAnswer:"
answer = openai.ChatCompletion.create(model="gpt-4o",
messages=[{"role": "user", "content": prompt}])
```

The retrieval ensures the model has up-to-date facts. You might also prepend a system message like "Use the context to answer exactly." for clarity.

- **Instruction-Tuning Example:**

In fine-tuning datasets (e.g. Alpaca), each example is a pair of {"instruction": ..., "input": ..., "output": ...}. For instance:

```
instruction: "Translate the following English text to German."  
input: "\"Good morning.\""  
output: "\"Guten Morgen.\""
```

This trains the model to follow instructions. For prompting alone, instruction tuning means the model already understands instructions (e.g. ChatGPT). We can still prime it:

```
"System: You are a translator that strictly outputs JSON.\nUser: Translate 'Hello' to German.\nAssistant: {\"translation\": \"Hallo\"}"
```

## 7. Evaluation and Testing

- **Automatic Metrics:** Use task-appropriate metrics (accuracy, F1 for classification; BLEU/ROUGE for translation/summarization; etc.). For open-ended tasks, use semantic similarity or F1 with reference data if available.
- **Human Evaluation:** For creative or subjective tasks, have humans rate outputs (coherence, relevance, correctness).
- **Calibration:** LLM confidence (via token probabilities) is often miscalibrated. Techniques like *self-consistency* (sampling multiple outputs) can give a pseudo-confidence (how often one answer occurs) [18†L49-L58]. Some work (Tang *et al.* 2023) explores getting the model to output explicit probabilities, but this is still research.
- **Adversarial Testing:** Intentionally include tricky inputs to test robustness (e.g. contradiction, ambiguous instructions, attempts at injection). Keep a suite of “red team” prompts. If output is unacceptable, add new instructions or fine-tune accordingly.
- **Regression Testing:** When prompt or model versions change, re-run important test queries to ensure performance hasn’t degraded (similar to software regression tests).

## 8. Safety and Robustness

Besides prompt injection, consider biases and harmful content. Use built-in content filters or moderation APIs to catch disallowed outputs. Prompt-level safety: explicitly instruct the model to refuse certain requests (e.g. “Do not provide medical advice”). However, such instructions can sometimes be overridden by users – hence defense-in-depth (RLHF, external filters, manual review for high-risk apps). Make sure to comply with legal/regulatory guidelines (e.g. GDPR, FDA) when using LLM outputs.



## 9. Engineering Checklist

When building a prompt-driven feature, consider:

- [ ] **Define the Task Clearly:** Know precisely what output format and content you need. Write a few sample inputs/desired outputs.
- [ ] **Start Simple:** Write a basic prompt (zero-shot). Test on sample inputs.
- [ ] **Iterate with Examples:** If needed, add 1–5 few-shot examples. Use relevant, representative cases. Check for *over-prompting* (too many examples).
- [ ] **Refine Instructions:** Make the prompt as explicit as possible. Use bullets, newlines, or delimiters to structure complex instructions [9†L39-L47] . Include *positive instructions* (what to do) rather than negatives [9†L130-L139] .
- [ ] **Choose Model & Parameters:** Based on needed accuracy, choose an LLM model. Set temperature/top-p: use low randomness for factual tasks [47†L198-L201] . Set `max_tokens` enough for output, but not too high to waste cost [47†L203-L211] .
- [ ] **Test and Validate:** Evaluate on a validation set. Measure metrics. Adjust prompt if errors. Check edge cases (e.g. missing context, ambiguous queries).
- [ ] **Implement Safeguards:** Add guardrail prompts or filters (e.g. “If uncertain, answer ‘I don’t know’” or double-check logic). Use moderation if needed.
- [ ] **Deploy with Monitoring:** Log usage (no PII). Monitor performance (latency, error rates). Set up alarms for failures.
- [ ] **Version & Document:** Keep track of prompt text and changes. Document the prompt logic and intended behavior for future maintainers.

## 10. Multimodal and Provider-Specific Notes

- **Multimodal Prompting:** Modern LLMs can take images or other modalities. For example, GPT-4 Vision (GPT-4V) can accept an image URL or base64 in the prompt with type `"image_url"` . A prompt might combine text and image parts, e.g.:

```
messages = [  
  {"role": "user", "content": [  
    {"type": "text", "text": "What objects are in this image?"},  
    {"type": "image_url", "image_url": {"url": "https://example.com/  
image.jpg"}}  
  ]}  
]
```

The model then describes the image. Similar patterns hold for audio or video inputs in capable models. Design prompts to clearly instruct how to process the image content.

- **OpenAI (ChatGPT, GPT API):** Use system messages for high-level instructions. The OpenAI “Best Practices” guide is authoritative [9†L39-L47] [9†L58-L66] . Note new API features: roles like “developer” and an `instructions` parameter for GPT-5 style models [80†L1338-L1346] . They support function-calling (for tool use) and structured JSON mode as shown above [74†L125-L134] .

- **Google Gemini:** Gemini's API allows JSON and function output via schema config [72†L19-L27] . It also supports tool use (search, code execution) combined with structured outputs [71†L1065-L1074] . The syntax uses `response_format` and `tools` in a `generateContent` call (examples in [71]). Prompting principles are similar: use clear instructions, examples, and specify output format. Google's docs emphasize using Pydantic/Zod to define schemas [72†L19-L27] .
- **Other APIs (Azure OpenAI, Claude, etc.):** The ideas carry over. Some use slightly different parameter names (e.g. Azure's OpenAI has same structure; Anthropic's Claude uses "system" vs "assistant"). Always refer to provider docs for details on roles, tokens, and capabilities.

## 11. Comparative Techniques

| Technique                       | Complexity     | Effectiveness         | Typical Use Cases                                    | Notes                                                                                     |
|---------------------------------|----------------|-----------------------|------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Zero-Shot Prompt</b>         | Low            | Medium                | Simple Q&A, single-turn tasks                        | Quick to implement; relies on model's knowledge.                                          |
| <b>Few-Shot Prompt</b>          | Medium         | High (if tuned)       | Classification, transformation tasks                 | More engineering (curating examples). Risk of <i>over-prompting</i> .                     |
| <b>Chain-of-Thought</b>         | Medium         | Very High (reasoning) | Math/logic puzzles, complex QA                       | Requires prompting with reasoning steps. Emergent with large models [16†L55-L64] .        |
| <b>Self-Consistency</b>         | High (compute) | High                  | Critical decision tasks                              | Run multiple CoT traces and aggregate answer [18†L49-L58] . Improves accuracy but slower. |
| <b>Prompt Chaining</b>          | Medium         | High (composed tasks) | Multi-step workflows (e.g. summarization → analysis) | Break tasks into stages; requires managing intermediate outputs.                          |
| <b>RAG (Retrieval)</b>          | High           | High (factual)        | Up-to-date/factual Q&A, long-term memory             | Needs vector store/search. Improves grounding [22†L52-L60] .                              |
| <b>Tool Use / Agents</b>        | Very High      | High (specialized)    | Tasks requiring calculation, code exec, search       | Complex engineering (API calls, orchestration). For extensibility.                        |
| <b>Structured (JSON) Output</b> | Low            | High (parsing)        | Any structured data extraction (forms, QA)           | Ensures valid schema [26†L98-L107] [72†L19-L27] . Requires supported models.              |

| Technique                      | Complexity     | Effectiveness               | Typical Use Cases                    | Notes                                                 |
|--------------------------------|----------------|-----------------------------|--------------------------------------|-------------------------------------------------------|
| <b>Prompt Tuning (learned)</b> | Very High      | Very High (domain-specific) | Specialized domains / repeated tasks | Fine-tune or prefix-tune models. Outside prompt text. |
| <b>Ensembling/Voting</b>       | High (compute) | Medium to High              | High-reliability answers             | Run multiple prompts/models, aggregate. Expensive.    |

Table 1: Comparison of prompt engineering techniques.

## 12. Visualization: Prompt Workflow

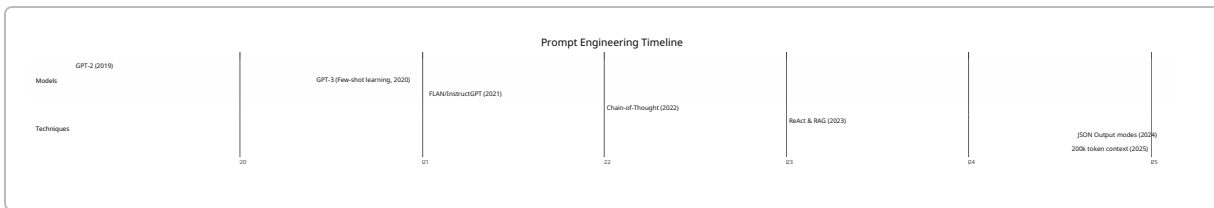


Figure 2: Milestones in prompt engineering (dates approximate).

## 13. Additional Resources

### Seminal Papers and Sources:

- Brown et al. (2020), "Language Models are Few-Shot Learners" [16†L55-L64] – introduced GPT-3 few-shot prompting.
- Wei et al. (2022), "Chain of Thought Prompting" [16†L55-L64] – key paper on CoT.
- Wang et al. (2022), "Self-Consistency" [18†L49-L58] – improving CoT via voting.
- Yao et al. (2023), "ReAct" [20†L53-L62] – LLM reasoning plus actions.
- Lewis et al. (2020), "Retrieval-Augmented Generation" [22†L52-L60] – RAG method.
- Schick et al. (2023), "Toolformer" [24†L54-L63] – LLM uses API tools.
- OpenAI Blog: "Introducing GPT-4" (2023) – outlines system/user messages, chat format, tool use, etc.
- OpenAI API Docs: *Prompt Engineering Guide* [9†L39-L47] [9†L58-L66] ; *Chat Completions API* (system/user roles, function calling).
- Google GenAI Docs: *Structured Output* guide [72†L19-L27] .
- IBM Think articles on prompts and AI safety: NIST prompt injection overview [83†L91-L100] [83†L132-L140] .
- LaunchDarkly *Prompt Versioning* blog [59†L111-L119] .
- OpenAI Cookbook: *Handling Rate Limits* [64†L1046-L1054] .

### Prompt Engineering Surveys and Guides:

- Sharma et al. (2023), "A Systematic Survey of Prompt Engineering Techniques" (arXiv) – comprehensive taxonomy (Table of Contents and sections were used above).

- Wei et al. (2024), “The Prompt Report” (arXiv) – meta-analysis with history (we cited history section).
- Prompt engineering books/blogs: *The Prompting Guide* (promptingguide.ai) covers many techniques; *Learn Prompting* (free ebook) covers fundamentals.

#### Checklist for Practitioners:

1. **Define goal:** Clarify what you want the LLM to do (task definition, output format).
2. **Choose model & interface:** Pick an API (Chat vs Completion) and model size appropriate to task.
3. **Draft prompt:** Start with a concise instruction or question.
4. **Test & refine:** Try on sample inputs. If wrong, iterate: adjust wording, add examples, constraints, or role.
5. **Control randomness:** Set temperature/top-p per task (0 for factual, >0 for creativity).
6. **Format output:** Provide output examples or JSON schemas to make parsing easier.
7. **Validate output:** Parse and check responses (e.g. with JSON validators or secondary LLM checks).
8. **Add safety checks:** Filter or re-prompt on disallowed content/hallucinations.
9. **Monitor and log:** Once in production, monitor metrics, log outputs, and retrain on failures.
10. **Version & experiment:** Keep prompt versions, use A/B testing on real users/data to optimize.

Following these guidelines and leveraging the cited techniques can help build effective, robust prompt-based applications.

**Sources:** We have drawn on the latest literature and provider documentation [9†L39-L47] [16†L55-L64] [18†L49-L58] [26†L98-L107] [72†L19-L27] [80†L1190-L1198] [83†L98-L106] [59†L111-L119] among others, and recommend consulting those references for deeper detail.

---